Key-Based Messaging

Implementation Guide

Stack: .NET 10 (LTS) · ASP.NET Core 10 · C# 14 · Angular 21 · TypeScript

A step-by-step guide for implementing a key-based alert messaging system using a .NET 10 backend and Angular 21 frontend. Backend and frontend tasks are separated so they can be built in parallel. By the end, your team will have a fully decoupled, i18n-ready notification pipeline.

Attribute	Detail
Backend	.NET 10 LTS · ASP.NET Core 10 · C# 14
Frontend	Angular 21 · TypeScript · Signals · Zoneless
Pattern	Key-based string protocol · pipe-delimited params
Transport	HTTP GET (responseType: 'text')
i18n	JSON template file with {{N}} placeholders
Test runner	Vitest (Angular 21 default)

Version-Specific Notes

Before writing any code, be aware of the following defaults that differ from earlier versions of each framework.

.NET 10

- .NET 10 is an LTS release supported until November 2028. Set TargetFramework to net10.0 in your .csproj.
- The minimal hosting model (WebApplication.CreateBuilder) is unchanged and is the correct approach. The legacy WebHost model was removed.
- IActionContextAccessor is now obsolete. Do not reference it. It is not used in this pattern.
- WithOpenApi() extension was deprecated. Use the new OpenAPI generator APIs if you need Swagger. Not needed for this guide.
- Cookie challenge redirects are disabled for API endpoints by default — they now return 401/403 directly. This is the correct behaviour for an API.

- C# 14 is the default language. All switch expressions, primary constructors, and nullable reference types used in this guide are fully supported.

Angular 21

- Zoneless change detection is now the default for new projects. Zone.js is no longer included. Do not add it. This guide's code is fully zoneless-compatible.
- HttpClient is included by default in new Angular 21 projects. provideHttpClient() is pre-configured in app.config.ts by the CLI. If you are migrating from an older project, verify it is present.
- *ngIf, *ngFor, and *ngSwitch are deprecated. Use the built-in @if, @for, and @switch control flow syntax instead. CommonModule is no longer needed for template control flow.
- Vitest is the default test runner. Karma and Jasmine are no longer generated for new projects. Tests written in this guide style work with Vitest out of the box.
- Signal Forms are available as a developer preview. This guide uses standard Signals for state management — Signal Forms are not required here.
- Standalone components are the default. NgModule is no longer generated by the CLI for new projects.

How the Pattern Works

The core idea is a structured string protocol. The backend returns a single plain-text string over HTTP. That string encodes a namespaced key and pipe-delimited parameters. The frontend parses the string, looks up a matching message template in a JSON file, substitutes the parameters, and renders the result.

```
Wire string (from backend):
  MONEY.TRANSFER_SUCCESS|12500|Priya M.|87340

Template (from en.json):
  "₹{{0}} has been sent to {{1}}. Your new balance is ₹{{2}}."

Resolved message (rendered in UI):
  ₹12500 has been sent to Priya M. Your new balance is ₹87340.
```

The key (MONEY.TRANSFER_SUCCESS) serves three purposes: it identifies what happened, determines which template to render, and implies the severity of the alert (INFO, WARNING, or ERROR) without the backend needing to send a separate field.

Backend Tasks (.NET 10 / C#)

These five steps can be completed independently of the frontend. The only shared contract is the wire string format agreed in Step B1.

B1 · Define the Wire String Format

Before writing any code, agree on the string format with the frontend team. Document it as a comment or README entry so both sides build to the same spec.

```
Format:  <NAMESPACE>.<EVENT_NAME>|<param0>|<param1>|...

Rules:
  • Namespace and event name are UPPER_SNAKE_CASE
  • Namespace and event are separated by a single dot (.)
  • Parameters are separated by pipes (|)
  • Parameters are always plain strings — no JSON, no quoting
  • No trailing pipe

Examples:
  LOGIN.LOGIN_SUCCESS|30
  MONEY.TRANSFER_SUCCESS|12500|Priya M.|87340
  LOGIN.USER_LOCKED|5
```

Contract Both teams must agree on this format before coding begins. Any change to parameter order is a breaking change for the frontend.

B2 · Define the Interface

Create an interface that the controller will depend on. This keeps the controller decoupled from the implementation and makes the service swappable for testing.

```
// Interfaces/IAlertService.cs
namespace YourApp.Interfaces;

public interface IAlertService
{
    string? GetAlert(int choice);
}
```

C# 14 File-scoped namespaces (namespace YourApp.Interfaces;) are the standard in .NET 10 projects. The guide uses this style throughout.

B3 · Implement the Service

The implementation returns the wire string for each alert type. For a demo or prototype, use a switch expression with hardcoded values. In production, replace the switch body with real data sources — database queries, event triggers, etc.

```
// Implementations/AlertService.cs
using YourApp.Interfaces;

namespace YourApp.Implementations;

public class AlertService : IAlertService
{
    public string? GetAlert(int choice) => choice switch
    {
        1 => "LOGIN.LOGIN_SUCCESS|30",
        2 => "LOGIN.WELCOME_USER|Subham|4",
        3 => "MONEY.TRANSFER_SUCCESS|12500|Priya M.|87340",
        4 => "MONEY.EMI_DUE|8200|3|05 May",
        5 => "MONEY.BALANCE_BELOW_THRESHOLD|1820|2500",
        6 => "MONEY.DAILY_LIMIT_REACHED|50000|7",
        7 => "LOGIN.LOGIN_ATTEMPTS_REMAINING|2",
        8 => "LOGIN.USER_LOCKED|5",
        _ => null
    };
}
```

Production note Replace the integer switch with a strongly-typed trigger — an enum or domain event. The integer works for demos but is implicit knowledge that breaks under refactoring.

B4 · Register the Service and Configure CORS

Register the interface-to-implementation mapping and configure CORS in Program.cs. In .NET 10 the minimal hosting model is the only supported pattern.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Register alert service
builder.Services.AddScoped<IAlertService, AlertService>();

// Allow Angular dev server to call the API
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngularDev", policy =>
        policy.WithOrigins("http://localhost:4200")
              .AllowAnyMethod()
              .AllowAnyHeader());
});

var app = builder.Build();
app.UseCors("AllowAngularDev");
app.MapControllers();
app.Run();
```

B5 · Create the Controller

The controller is intentionally thin — it only receives the request, delegates to the service, and returns the raw string as plain text. No JSON serialisation needed.

```
// Controllers/AlertController.cs
using Microsoft.AspNetCore.Mvc;
using YourApp.Interfaces;

namespace YourApp.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AlertController(IAlertService alertService) : ControllerBase
{
    [HttpGet("{choice:int}")]
    public IActionResult Get(int choice)
    {
        var result = alertService.GetAlert(choice);
        if (result is null) return NotFound();

        // Return plain text — Angular reads this with responseType: 'text'
        return Content(result, "text/plain");
    }
}
```

C# 14 — Primary constructor AlertController(IAlertService alertService) is a primary constructor. The injected service is available directly without a private field assignment. This is the idiomatic .NET 10 style.

Test it Run the project and GET /api/alert/3. You should see the raw string: MONEY.TRANSFER_SUCCESS|12500|Priya M.|87340

Frontend Tasks (Angular 21 / TypeScript)

These six steps can be completed independently once the wire format from B1 is agreed. Use triggerMock() (Step F3) to develop and test all templates without a running backend.

F1 · Set Up the Environment Config

Add the backend URL to your Angular environment files. Angular 21 CLI generates these by default when you use --environments flag, or create them manually.

```
// src/environments/environment.ts
export const environment = {
  production: false,
  backendAlertUrl: 'http://localhost:5000/api/alert'
};

// src/environments/environment.prod.ts
```

```
export const environment = {
  production: true,
  backendAlertUrl: 'https://your-production-api.com/api/alert'
};
```

F2 · Create the Model and Parser

Create src/app/models/api-response.model.ts. This file owns the ParsedAlert shape, the Severity type, the severity map, and the parser function. All parsing logic lives here and nowhere else.

```
// src/app/models/api-response.model.ts

export type Severity = 'INFO' | 'WARNING' | 'ERROR';

// Severity derived from key namespace — backend does not send it
const SEVERITY_MAP: Record<string, Severity> = {
  'LOGIN.LOGIN_SUCCESS':             'INFO',
  'LOGIN.WELCOME_USER':              'INFO',
  'MONEY.TRANSFER_SUCCESS':          'INFO',
  'MONEY.EMI_DUE':                   'WARNING',
  'MONEY.BALANCE_BELOW_THRESHOLD':   'WARNING',
  'MONEY.DAILY_LIMIT_REACHED':       'WARNING',
  'LOGIN.LOGIN_ATTEMPTS_REMAINING': 'ERROR',
  'LOGIN.USER_LOCKED':               'ERROR',
};

export interface ParsedAlert {
  messageKey: string;                // e.g. 'LOGIN.WELCOME_USER'
  params: Record<string, string>;    // e.g. { '0': 'Subham', '1': '4' }
  severity: Severity;
  raw: string;                       // original string for debug panel
}

export function parseAlertString(raw: string): ParsedAlert | null {
  if (!raw?.trim()) return null;

  const parts = raw.split('|');
  const messageKey = parts[0].trim();
  const positionalValues = parts.slice(1);

  const params = Object.fromEntries(
    positionalValues.map((val, idx) => [String(idx), val.trim()])
  );

  const severity = SEVERITY_MAP[messageKey] ?? 'INFO';
  return { messageKey, params, severity, raw };
}
```

F3 · Create the Alert Service

Create the Angular service. It uses Signals for reactive state — any component that reads alertService.alert() automatically updates when the signal changes. The triggerMock() method lets you test all templates without a running backend.

```
// src/app/services/alert.service.ts
import { inject, Injectable, signal } from '@angular/core';
import { parseAlertString, ParsedAlert } from '../models/api-response.model';
import { catchError, map, of } from 'rxjs';
import { HttpClient } from '@angular/common/http';
import { environment } from '../../environments/environment';

@Injectable({ providedIn: 'root' })
export class AlertService {
  private http = inject(HttpClient);

  alert   = signal<ParsedAlert | null>(null);
  loading = signal(false);
  error   = signal<string | null>(null);

  fetchAlert(choice: number): void {
    this.loading.set(true);
    this.error.set(null);

    this.http
      .get(`${environment.backendAlertUrl}/${choice}`, { responseType: 'text' })
      .pipe(
        map((raw) => parseAlertString(raw)),
        catchError(() => {
          this.error.set('Failed to fetch alert');
          return of(null);
        })
      )
      .subscribe((parsed) => {
        this.alert.set(parsed);
        this.loading.set(false);
      });
  }

  // Use during development to test templates without the backend
  triggerMock(raw: string): void {
    this.alert.set(parseAlertString(raw));
  }
}
```

HttpClient in Angular 21 HttpClient is included by default in new Angular 21 projects — provideHttpClient() is pre-configured in app.config.ts by the CLI. If migrating from an older project, verify it is present in app.config.ts.

F4 · Create the i18n Template File

Create src/assets/i18n/en.json. Each key corresponds exactly to a messageKey from the parser. Parameters are referenced by their positional index using {{N}} placeholders — the N matches the numeric key in the params object.

```
{
  "LOGIN": {
    "LOGIN_SUCCESS": "Authenticated successfully. Session expires in {{0}}
minutes.",
```

```
    "WELCOME_USER": "Welcome back, {{0}}! You have {{1}} new alert(s).",
    "LOGIN_ATTEMPTS_REMAINING": "{{0}} attempt(s) remaining before your account is
locked.",
    "USER_LOCKED": "Your account has been locked after {{0}} failed attempts."
  },
  "MONEY": {
    "TRANSFER_SUCCESS": "₹{{0}} has been sent to {{1}}. Your new balance is
₹{{2}}.",
    "EMI_DUE": "Your EMI of ₹{{0}} is due in {{1}} day(s) on {{2}}.",
    "BALANCE_BELOW_THRESHOLD": "Your balance ₹{{0}} is below the minimum ₹{{1}}.",
    "DAILY_LIMIT_REACHED": "Daily limit of ₹{{0}} reached. {{1}} transaction(s)
today."
  }
}
```

Important Every key in the backend's AlertService must have a matching entry here. A missing key causes renderAlert() to fall back to the raw string.

F5 · Create the Renderer Utility

Create a utility function that resolves a ParsedAlert into a human-readable string. It navigates the JSON by splitting messageKey on '.' and interpolates parameters by replacing {{N}} with the matching param value.

```
// src/app/utils/alert-renderer.ts
import { ParsedAlert } from '../models/api-response.model';
import translations from '../../assets/i18n/en.json';

export function renderAlert(alert: ParsedAlert): string {
  const [namespace, event] = alert.messageKey.split('.');

  const ns = (translations as any)[namespace];
  if (!ns) return alert.raw;

  const template: string = ns[event];
  if (!template) return alert.raw;

  // Replace {{0}}, {{1}}, {{2}} with matching params
  return template.replace(
    /\{\{(\d+)\}\}/g,
    (_, idx: string) => alert.params[idx] ?? ''
  );
}
```

F6 · Build the Alert Component

Create the display component. In Angular 21 use @if instead of *ngIf, and do not import CommonModule — it is not needed for built-in control flow syntax.

```
// src/app/components/alert-display/alert-display.component.ts
import { Component, inject, computed } from '@angular/core';
import { AlertService } from '../../services/alert.service';
```

```
import { renderAlert } from '../../utils/alert-renderer';

// Note: no CommonModule import needed in Angular 21
// @if / @else are built into the template engine

@Component({
  selector: 'app-alert-display',
  standalone: true,
  imports: [],
  template: `
    @if (alertSvc.loading()) {
      <div class='alert loading'>Loading...</div>
    }
    @if (alertSvc.error()) {
      <div class='alert error'>{{ alertSvc.error() }}</div>
    }
    @if (alertSvc.alert()) {
      <div [class]='severityClass()'>{{ message() }}</div>
    }
  `
})
export class AlertDisplayComponent {
  alertSvc = inject(AlertService);

  message = computed(() => {
    const a = this.alertSvc.alert();
    return a ? renderAlert(a) : '';
  });

  severityClass = computed(() => {
    const severity = this.alertSvc.alert()?.severity ?? 'INFO';
    return `alert ${severity.toLowerCase()}`;
  });
}
```

Angular 21 — @if vs *ngIf *ngIf is deprecated. Always use @if in Angular 21. It requires no import, no CommonModule, and works correctly with the default zoneless change detection.

Wiring It Together

Parent Component

Inject AlertService and call fetchAlert() for real data or triggerMock() during development.

```
// src/app/app.component.ts
import { Component, inject } from '@angular/core';
import { AlertService } from './services/alert.service';
import { AlertDisplayComponent } from './components/alert-display/alert-
display.component';

@Component({
  selector: 'app-root',
  standalone: true,
```

```
  imports: [AlertDisplayComponent],
  template: `
    <app-alert-display />
    <button (click)='load(3)'>Transfer Success</button>
    <button (click)='load(5)'>Low Balance</button>
    <button (click)='load(8)'>Account Locked</button>
  `
})
export class AppComponent {
  private alertSvc = inject(AlertService);
  load(choice: number) { this.alertSvc.fetchAlert(choice); }
}
```

Full End-to-End Trace

```
1. User clicks 'Transfer Success'
   → AppComponent calls alertSvc.fetchAlert(3)

2. AlertService sets loading = true
   → GET http://localhost:5000/api/alert/3

3. .NET 10 AlertController receives request
   → AlertService.GetAlert(3) returns "MONEY.TRANSFER_SUCCESS|12500|Priya M.|87340"
   → Controller returns Content("...", "text/plain")

4. Angular receives raw text, calls parseAlertString(raw)
   → messageKey = 'MONEY.TRANSFER_SUCCESS'
   → params     = { '0': '12500', '1': 'Priya M.', '2': '87340' }
   → severity   = 'INFO'

5. AlertService sets alert signal with ParsedAlert
   → loading = false
   → Zoneless change detection picks up signal change instantly

6. AlertDisplayComponent re-renders (no zone.js tick needed)
   → renderAlert() → en.json → MONEY → TRANSFER_SUCCESS
   → "₹12500 has been sent to Priya M. Your new balance is ₹87340."
   → CSS class: 'alert info'
```

Adding a New Alert Type

Adding a new alert type requires changes in exactly three places, with no modifications to existing code.

File	Change required
AlertService.cs (backend)	Add one new case to the switch expression
api-response.model.ts (frontend)	Add the key and its Severity to SEVERITY_MAP
en.json (frontend)	Add the key and its message template

Example — adding a MONEY.REFUND_PROCESSED alert:

```
// Backend: AlertService.cs — add one case
9 => "MONEY.REFUND_PROCESSED|3500|Amazon|15 May",

// Frontend: api-response.model.ts — add to SEVERITY_MAP
'MONEY.REFUND_PROCESSED': 'INFO',

// Frontend: en.json — add the template inside MONEY block
"REFUND_PROCESSED": "₹{{0}} refund from {{1}} will be credited by {{2}}."
```

Zero breaking changes AlertDisplayComponent, AlertService, renderAlert(), and parseAlertString() require no modifications. Only the three data locations change.

Pre-Launch Checklist

Backend (.NET 10)

- TargetFramework is net10.0 in the .csproj
- IAlertService interface defined with file-scoped namespace
- AlertService registered as Scoped in Program.cs
- Controller returns Content(result, "text/plain") — not JSON
- Route constraint uses {choice:int} to reject non-integer inputs
- CORS policy permits the Angular dev server origin (http://localhost:4200)
- GET /api/alert/{choice} returns 404 for unknown choice

Frontend (Angular 21)

- provideHttpClient() is present in app.config.ts (pre-included in new Angular 21 projects)
- All templates use @if / @for / @switch — no *ngIf or *ngFor
- CommonModule is not imported in components that only use built-in control flow
- provideZonelessChangeDetection() is in app.config.ts (default for new Angular 21 projects)
- environment.ts has backendAlertUrl set correctly
- parseAlertString handles null/empty input without throwing
- Every backend key is present in SEVERITY_MAP
- Every backend key is present in en.json
- renderAlert() falls back to alert.raw if template not found
- All templates tested with triggerMock() before backend integration

Integration

- End-to-end trace verified for at least one INFO, WARNING, and ERROR alert

- Network tab confirms response is plain text — not JSON
- CSS classes (info / warning / error) styled appropriately
- Zoneless change detection confirmed working — no manual detectChanges() calls

Common Mistakes to Avoid

Mistake	Fix
Using *ngIf in Angular 21 templates	Use @if instead. *ngIf is deprecated and will be removed. CommonModule is not needed for @if.
Returning JSON from the controller	Use Content(result, "text/plain"). Angular reads this with responseType: 'text'. JSON causes a parse error.
Changing parameter order in the backend string	The frontend uses positional indices. param[0] becoming param[1] silently renders wrong data. Treat parameter order as a versioned API contract.
Adding a backend key but not updating en.json	renderAlert() falls back to the raw string. Add the template to en.json at the same time as the backend case.
Adding zone.js to an Angular 21 project	New Angular 21 projects are zoneless by default. Do not add zone.js. Signals with @if work correctly without it.
Forgetting responseType: 'text' in HttpClient	Without it, Angular attempts to parse the plain-text response as JSON and throws a SyntaxError. Always specify responseType: 'text' for this endpoint.